

Kiran Fatima (23-SE-05)
WE Lab Report no. 6

Lab Exercise 1:

Code:

```
// 1. Create an object named 'person' with given properties
```

```
let person = {  
  firstName: "Kiran",  
  lastName: "Fatima",  
  Roll_no: "22-SE-45",  
  city: "Taxila"  
};
```

```
// 2. Print firstName and lastName using object properties
```

```
console.log("First Name:", person.firstName);  
console.log("Last Name:", person.lastName);
```

```
// 3. Add a new property 'country' to the person object
```

```
person.country = "Pakistan";
```

```
// 4. Print the updated person object
```

```
console.log("Updated Person Object:", person);
```

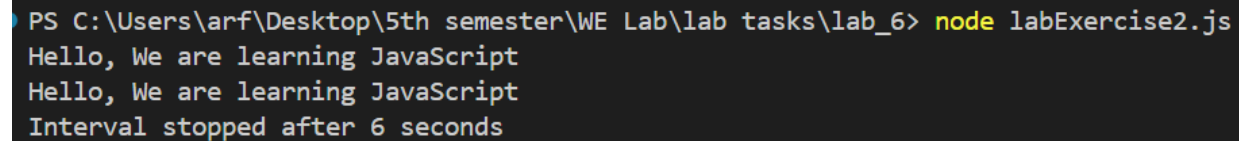
Output:

```
PS C:\Users\arf\Desktop\5th semester\WE Lab\lab tasks\lab_6> node labExercise1.js  
First Name: Kiran  
Last Name: Fatima  
Updated Person Object: {  
  firstName: 'Kiran',  
  lastName: 'Fatima',  
  Roll_no: '22-SE-45',  
  city: 'Taxila',  
  country: 'Pakistan'  
}
```

Lab Exercise 2:

Code:

```
// Function to print message
function showMessage() {
  console.log("Hello, We are learning JavaScript");
}
// Call the function every 2 seconds using setInterval()
let intervalID = setInterval(showMessage, 2000);
// Stop the interval after 6 seconds using setTimeout()
setTimeout(function() {
  clearInterval(intervalID);
  console.log("Interval stopped after 6 seconds");
}, 6000);
```

Output:A terminal window with a dark background. The prompt is 'PS C:\Users\arf\Desktop\5th semester\WE Lab\lab tasks\lab_6>'. The command 'node labExercise2.js' has been executed. The output consists of three lines: 'Hello, We are learning JavaScript', 'Hello, We are learning JavaScript', and 'Interval stopped after 6 seconds'.

```
PS C:\Users\arf\Desktop\5th semester\WE Lab\lab tasks\lab_6> node labExercise2.js
Hello, We are learning JavaScript
Hello, We are learning JavaScript
Interval stopped after 6 seconds
```

Lab Exercise 3:**Code:**

```
// Using Promises
function promiseFunction() {
  // Create a new Promise
  return new Promise((resolve, reject) => {
    // Simulate an async operation (e.g., API call, file read)
    setTimeout(() => {
      resolve("Resolved");
    }, 2000); // Simulate delay
  })
}
```

```
console.log("Start");

promiseFunction()

.then((result) => { console.log(result);

console.log("End"); })

.catch((error)=>{console.log(error)

});
```

Output:

```
PS C:\Users\arf\Desktop\5th semester\WE Lab\lab tasks\lab_6> node labExercise3.js
Start
Resolved
End
```

"Start" appears immediately.

"Resolved" and "End" appear 2 seconds later.

Lab Exercise 4:

Code:

```
async function loadData() { const users = await Promise.all([
fetch('https://jsonplaceholder.typicode.com/users/1').then(res => res.json()),
fetch('https://jsonplaceholder.typicode.com/users/2').then(res => res.json()),
fetch('https://jsonplaceholder.typicode.com/users/3').then(res => res.json()) ]);
console.log("Users:", users); } loadData();
```

Output:

```
Users: [
  {
    id: 1,
    name: 'Leanne Graham',
    username: 'Bret',
    email: 'Sincere@april.biz',
    address: {
      street: 'Kulas Light',
      suite: 'Apt. 556',
      city: 'Gwenborough',
      zipcode: '92998-3874',
      geo: [Object]
    },
    phone: '1-770-736-8031 x56442',
    website: 'hildegard.org',
    company: {
      name: 'Romaguera-Crona',
      catchPhrase: 'Multi-layered client-server neural-net',
      bs: 'harness real-time e-markets'
    }
  },
]
```

```
{
  id: 2,
  name: 'Ervin Howell',
  username: 'Antonette',
  email: 'Shanna@melissa.tv',
  address: {
    street: 'Victor Plains',
    suite: 'Suite 879',
    city: 'Wisokyburgh',
    zipcode: '90566-7771',
    geo: [Object]
  },
  phone: '010-692-6593 x09125',
  website: 'anastasia.net',
  company: {
    name: 'Deckow-Crist',
    catchPhrase: 'Proactive didactic contingency',
    bs: 'synergize scalable supply-chains'
  }
},
```

```
{
  id: 3,
  name: 'Clementine Bauch',
  username: 'Samantha',
  email: 'Nathan@yesenia.net',
  address: {
    street: 'Douglas Extension',
    suite: 'Suite 847',
    city: 'McKenziehaven',
    zipcode: '59590-4157',
    geo: [Object]
  },
  phone: '1-463-123-4447',
  website: 'ramiro.info',
  company: {
    name: 'Romaguera-Jacobson',
    catchPhrase: 'Face to face bifurcated interface',
    bs: 'e-enable strategic applications'
  }
}
]
```

The output of 1st three users from the given API will be shown on node.js console.

Lab Exercise 5:

Code:

// Function that returns a Promise

```
function checkmail() {  
  return new Promise((resolve, reject) => {  
    let randomNumber = Math.floor(Math.random() * 10) + 1; // Generates number 1–10  
    console.log("Random number:", randomNumber);  
  
    if (randomNumber > 5) {  
      resolve("Mail received successfully!");  
    } else {  
      reject("Failed to arrive");  
    }  
  });  
}
```

// Calling the function

```
checkmail()  
  .then((mail) => {  
    console.log(mail); // If promise is resolved  
  })  
  .catch((error) => {  
    console.log(error); // If promise is rejected  
  })  
  .finally(() => {  
    console.log("Experiment completed");  
  });
```

Output:

When random number is greater than 5:

```
Random number: 6
Mail received successfully!
Experiment completed
```

When random number is less than 5:

```
Random number: 2
Failed to arrive
Experiment completed
```

Lab Task 1:

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Random Quotes on Page Refresh</title>
  <style>
    body {
      font-family: Arial, Helvetica, sans-serif;
      margin: 40px;
    }
    h1 {
      font-size: 24px;
      font-weight: bold;
    }
    h2 {
      font-size: 18px;
      font-weight: bold;
    }
    hr {
```

```

    margin: 15px 0;
}
p{
    font-size: 16px;
}
em{
    color: #333;
}
</style>
</head>
<body>
    <h1>Random Quotes on Page Refresh</h1>
    <h2>Random quotes</h2>
    <hr>
    <p id="quote"></p>
    <p><em id="author"></em></p>

    <script>
        // Array of quotes and authors
        const quotes = [
            { text: "Every time history repeats itself the price goes up.", author: "Anonymous" },
            { text: "The moment you think you understand a great work of art, it's dead for you.",
              author: "Robert Wilson" },
            { text: "With faith, discipline and selfless devotion to duty, there is nothing worthwhile
              that you cannot achieve.", author: "Quaid e Azam" },
            { text: "Every violation of truth is not only a sort of suicide in the liar, but is a stab at the
              health of human society.", author: "Ralph Waldo" },
            { text: "Man is to be found in reason, God in the passions.", author: "G. C. Lichtenberg" },
            { text: "Great innovations should not be forced on slender majorities.", author: "Thomas
              Jefferson" },
            { text: "In this world nothing can be said to be certain, except death and taxes.", author:
              "Benjamin Franklin" },

```

```
{ text: "We have no more right to consume happiness without producing it than to  
consume wealth without producing it.", author: "George Bernard" },  
  
{ text: "So little done, so much to do.", author: "Cecil Rhodes" },  
  
{ text: "Nine-tenths of wisdom consists in being wise in time.", author: "Theodore  
Roosevelt" }  
  
];
```

```
// Generate random index using Math.random() and Math.floor()  
let randomIndex = Math.floor(Math.random() * quotes.length);  
  
// Select a random quote and author  
let selectedQuote = quotes[randomIndex];  
  
// Display on webpage  
document.getElementById("quote").textContent = selectedQuote.text;  
document.getElementById("author").textContent = "- " + selectedQuote.author;  
</script>  
</body>  
</html>
```

Output:

Random quotes are generated each time the page is refreshed.

Random Quotes on Page Refresh

Random quotes

Every time history repeats itself the price goes up.

- Anonymous

Random Quotes on Page Refresh

Random quotes

In this world nothing can be said to be certain, except death and taxes.

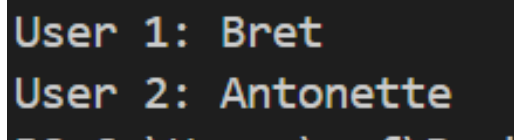
- Benjamin Franklin

Lab Task 2:

Code:

```
async function fetchUsers() {  
  try {  
    const response = await fetch("https://jsonplaceholder.typicode.com/users");  
    const users = await response.json();  
    console.log("User 1:", users[0].username);  
    console.log("User 2:", users[1].username);  
  }  
  catch (error) {  
    console.log("Error fetching data:", error);  
  }  
}  
fetchUsers();
```

Output:



```
User 1: Bret  
User 2: Antonette
```